# Method selection and planning

Group 9 - Inside No.9:

- Sky Burrows
- Oli Quantrill-Scott
- Daniel Basset
- Erikas Jovaisa
- James Garfield
- Oliver Lockett

# Outline & justification of the team's software engineering methods.

This team chose to follow a "waterfall" model for our software engineering model. This was considered over an "agile" model due to the limited time we found ourselves with after receiving our client brief, having the ability to outline a task, define it's approximate length and set a deadline for its completion has given us distinct milestones within the project so we can quantify our progress in a way that is meaningful to the organisation for those leading the project. We perhaps would have used an "agile" model if this project had the benefit of a looser time schedule, but faced with the hard deadline of fewer than 7 weeks to fully plan, implement and complete a project of this magnitude with our skill set and knowledge, we ultimately have set in motion a project model that will allow us to complete our objectives sequentially and definitely, with more time allocated to tasks individually but ultimately resulting in requiring less time to revise these implementations.

To implement this, we followed a system where we have a GANTT chart which is revised each week after its creation to adequately reflect on our progress and milestones, and use that data to project how long we imagine the tasks within the project will take. The GANTT chart also allows us to visualise the distinction between the phases of the project (requirements, design, implementation, etc..), something key to the "waterfall" model's project lifecycle.

A number of tools were considered for the various aspects of the project's development, but we ultimately decided to use GitHub for storing the project files, LibGDX as our game engine, Discord for team communication, and PlantUML for design of GANTT charts.

GitHub (https://github.com) was chosen over alternatives like GitLab and Azure for its simplicity and wide use. There is plenty of documentation for GitHub, and we anticipate that future engineers inheriting our project in the second phase will, at worst, be tentatively familiar with the software and, at best, have used it themselves.

While GitLab and Azure offered almost as many benefits as GitHub, GitLab was discounted because it is less well-known, and Azure was rejected as it requires a paywall, after a 30-day free trial, which the team was not comfortable expending. GitHub fits our Waterfall model well, as it can be used to track milestones and updates for a more definite timeline projection.

Using our initial game direction ideas (elicited from our Requirements) as a guide, we researched game engines that would be effective for developing a 2D, top-down game in Java. We settled on LibGDX primarily for its:

- Low overhead compared to alternatives: Beneficial for rendering many geese.
- Cross-platform support: This accommodates members of our team on Windows and Mac.
- Library support for pathfinding, suited for a game featuring goose enemies.

-Rejected Alternatives:

- jMonkeyEngine: Being less mainstream, there are fewer online tutorials.
- Godot Engine: Does not have Java as a native language.
- LWJGL: Rejected because it would require manually writing a window manager, collision system, renderer, pathfinding and asset loaders, before getting to coding our mechanics.

LibGDX also had benefits similar to GitHub. Of the game engines we looked into, it was the most reputable and well documented. This sped up our learning curve because there were lots of high-quality, accessible documentation and online bug fix resources. This also makes

our project easier to pick up for future engineers compared to other software we discussed, such as jMonkeyEngine.

The option of using no game engine, and instead implementing the project using the base java software and packages, was discussed. However, we ultimately determined that future developers taking over the project were more likely to recognise a standard game engine than a bespoke codebase. Adopting an established engine is also good practice for a long term, maintainable project.

Additionally, LibGDX aligned well with our chosen software engineering method. While developing without a game engine would have granted more control over low-level implementation, LibGDX provided greater benefits: It enabled easier error catching and debugging, resulting in fewer revisions, an advantage that better fit our Waterfall development model.

Discord (https://discord.com) was chosen as our communications channel over alternatives such as Slack and WhatsApp due to all team members already having the software, and its ability to have multiple communication channels for different purposes. WhatsApp does have a Communities feature, but we determined it would just be easier for people to join a Discord server than have to exchange phone numbers, as in the first weeks of the project we did not have any meeting in which all team members attended the meetings. This system of communication allowed for us to have dedicated sections for scheduling so we could ensure milestones were met within the right time frames, further benefiting our “waterfall” method.

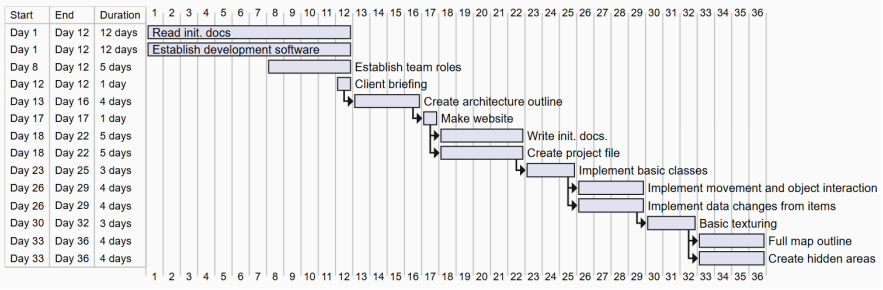
PlantUML was chosen as the tool for GANTT charts over the discussed alternative Lucidchart, due to how precisely we could manipulate data and its ease of allowing us to do so. Lucidchart was a tempting alternative, but it was determined the software interface allowed for too many things, too “noisy” in choices, that using a more dedicated software would be more convenient. This was the most beneficial to our “waterfall” method of all tools we used, as it allowed for the distinct visualisation of milestones, tasks and how they would fit into our schedule.

## Team’s approach to team organisation

The team had a loose approach to team organisation, with little distinction between the role types. This was because we determined towards the start of the project that we mostly fill the same role types as each other. That being said, one member of the team, alongside the work assigned to the group at large, acted slightly more in a directorial or administrative role, creating and maintaining the website, assigning people to different tasks that needed to be done, booking meeting rooms and leading our meetings. The system for assigning work was covering it all in the meeting, seeing who was available for what work and when, distinguishing the sub-tasks that needed to happen by the next milestone and assigning (typically 2) people to head that task and see it completed.

This approach is appropriate for the team as each of the members operate in a similar role type and skill set, therefore tasks are assigned primarily by who is available to work. The team started at very similar points, with nobody having any experience in LibGDX but most of us having some experience in Java and game design and creation. This approach is appropriate for the project as we have a directing force outlining the tasks that need to be done within the project, but we have a force of 7 equally abled individuals who are capable of performing the determined sub-tasks. The software engineering method the project is following, the “waterfall” method, defines a structure which uses a predetermined projection of tasks and how long they will take to complete; given this, having a administrative presence who can direct the team members to do a certain item of work and when to finish by fits very well within that structure.

# Systematic plan for the project




The first things we planned to accomplish were:
-Reading through the initial documents we were provided with e.g.(Product Brief) and,
-Establishing the development software we were going to use.
Both of these were started on day 1 and ran until day 12, as many of the subsequent tasks that needed doing, (such as the writing of the initial documentation and our overall plan for development which plays into the majority of the latter tasks), depended upon a thorough understanding of the project requirements and establishing our development software. We needed to establish the software (LibGDX) early to allow the team time to properly familiarise themselves with the software and potentially look for a replacement should any issues arise, which luckily there wasn't.

Between days 8 and 12, we also planned to establish our roles within the team and draw up a plan for our client briefing which was scheduled for day 12. We drew up some preliminary questions and what we were looking to get out of each question and established who within the group was going to be attending said meeting, as we were limited by office size as to how many members of the team could attend. The choices we would make in our architecture outline and many of the key features of the game, including the fundamental behaviour of the geese and how damage would be treated, were all dependent on what our client was looking for, which we could only realistically find out from this meeting.

From day 13 through till 16, we only planned to create our architecture outline sorted as all of the programming work we had coming up in the next 2 weeks would depend on us knowing how we wanted to organise our classes and dependencies and thus this was given a lot of priority and attention. However, following on from our client briefing, we came to the conclusion that we needed to spend some extra time on writing up the initial documentation, as it turned out to be a larger task than first anticipated, so we extended it from day 13 to day 22. Luckily, we made this adjustment to the original plan before that became an issue and overlapped with other important tasks. In the end we worked on both the architecture and the documentation simultaneously to ensure we had both done ahead of time as we wanted to avoid a situation where we had to worry about coding and writing up documents at the same time.

After the creation of the website, from day 18 to 22, we also needed to initially generate our project in LibGDX as without getting this out the way and ensuring everybody was able to create and edit files etc, we wouldn't be able to get to any of the coding and thus pretty much everything development related was dependent upon this hence why a relatively simple task was given a lot of time in the initial plan and this turned out to be an appropriate precaution.